

How to leverage Foundation Models for Embodied Agents

A survey on Embodied Agents

Jan Schweikert
yandev@gmx.de

Abstract

With the rapid development of LLMs, their applications have expanded far beyond conversational agents. Since the "boom" of LLMs, research in the field of embodied agents has leveraged such models to improve agents for generalizability. By leveraging the vast, internet-scale knowledge in LLMs, these agents have become significantly more generalizable and capable of understanding complex instructions. In this survey, we will discover different approaches of LLMs used in embodied agents as well as depicting the latest state-of-the-art models.

The introduction of LLMs has paved the way for agents to take on more complex tasks due to the rich semantic understanding as well as world knowledge they convey. This enables embodied agents which leverage LLMs to understand open-ended natural language instructions and disambiguate ambiguous instructions like "pick the red one" in a cluttered scene. For reasoning and planning, they decompose high-level goals into executable sub-goals. Additionally, their in-context learning and adaptability allow them to adjust to new tools, APIs, or environments with a small number of examples.

1 Introduction

The goal of creating autonomous agents that can perform a wide variety of tasks on behalf of humans has been a long-standing pursuit in the field of artificial intelligence (Wooldridge & Jennings, 1995; Wang et al., 2023). Within this goal, embodied agents occupy a compelling niche. These are AI systems that operate mainly within a physical environment to perform instructed tasks. The ability of embodied agents to observe, reason, and act in the real world offers a promising path toward general-purpose robotic intelligent systems in a wide variety of domains such as health-care, logistics or most common: domestic assistance.

Prior to the rise of LLMs, they were restricted in performing open-ended tasks because of limited generalization across environments, language understanding, and semantic grounding. These limitations have restricted embodied agents to perform in limited domains as well as to complete a limited number of tasks. (Alomari et al., 2017; Fang et al., 2019; Nguyen et al., 2020). Recent advancements in foundation models, particularly LLMs have marked a paradigm shift. (Vaswani et al., 2017; Brown et al., 2020; OpenAI, 2024)

This survey provides a technical overview of recent work in LLM-powered embodied agents, with a particular focus on the architectures and capabilities of such systems. Section 2 begins with the foundation concepts and architectures underlying LLMs. Section 3 covers a few approaches of how foundation models are integrated into the embodied agent framework, as well as benchmarks, and evaluation metrics. Section 4 discusses similarities and differences across approaches and common challenges. Finally, Section 5 concludes with key findings and the future potential of these systems.

2 Background

2.1 Large Language Models (LLMs)

LLMs are built on the Transformer architecture (Vaswani et al., 2017), a type of neural network that processes sequences in parallel rather than sequentially. The original Transformers architecture consists of an encoder-decoder structure, though many LLMs only use the decoder portion. (Brown et al., 2020; OpenAI, 2024; Touvron et al., 2023)

The key innovation of this architecture is how attention is kept across sentences. This is called "self-attention" and allows each token in a sequence to attend to all others. Unlike

previous recurrent architectures that processed text sequentially (Jordan et al., 1986; Hochreiter et al., 1997), self-attention enables the model to directly access information from any position in the input, allowing it to capture long-range dependencies more effectively. The architecture also utilizes multi-head attention, where multiple attention mechanisms run in parallel, allowing the model to focus on different types of relationships simultaneously. Additional components include positional encoding, which provide information about token positions, feed-forward networks within each layer, and residual connections with layer normalization. This architecture scales efficiently with model size and data, which leads to improvements in both language understanding and generation.

Training of LLMs is typically a multi-stage process. First, pre-training involves unsupervised learning on a large corpus of text, often internet-scale data. During this training phase, the model learns a broad set of skills and knowledge across a variety of domains. The model’s primary objective is to learn to predict the next token in a given sequence. Second, the pre-trained model is fine-tuned. This can be done by doing supervised fine-tuning (SFT), where the model is trained on instruction-following datasets to learn to respond helpfully to user queries (Wei et al., 2021). Third, many modern systems like GPT-4 employ Reinforcement Learning from Human Feedback (RLHF) (Ouyang et al., 2022), which fine-tunes the model based on preferences from human evaluators, who rank different model outputs in order to make the model more helpful and harmless.

In contrast, the original GPT-3 model (Brown et al., 2020) did not undergo any fine-tuning process. GPT-3 relied entirely on in-context learning like few-shot learning, where the model is given few examples as part of the prompt at inference time. This method does not update the model’s weights at all after training it.

2.2 Tying LLMs into Agent Frameworks

Embodied agents are systems that perceive and act in an environment, following a sense-plan-act loop. They observe the environment through sensors (vision, object detectors, or other modalities), generate a plan based on the current state and a given instruction, and decide on an action that modifies the environment. They operate in a

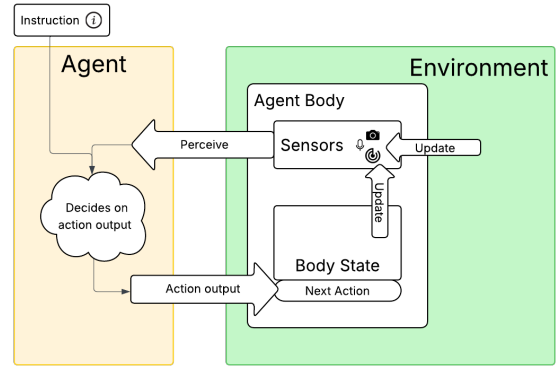


Figure 1: Embodied Agents Framework: the agent collects data through sensors like coordinates or camera images. The agent then decides on an action output, given an instruction and the current environment it is in. This process is typically iterated until the instruction or goal is completed and an end sequence token is decoded from the robot’s output

feedback loop as they observe the environment, then decide what to do next and act, which is also referred to as "closed-loop control". (Figure 1)

In many earlier LLM-powered agents, the language model typically acts as the "brain" but does not directly interact with the robot itself. Instead, it generates structured outputs, such as high-level plans, API calls, or code. These outputs are then sent to a separate low-level controller that translates them into the specific commands needed for the robot platform. However, more recent architectures incorporate multimodal models such as Vision-Language-Action (VLA), models which are designed to directly produce robot actions.

Depending on the architecture, the integration between the LLM and the robot platform can take different forms. In subgoal-planning agents, the LLM generates sub-goals from a natural-language instruction, which is passed to a low-level controller (Ahn et al., 2022; Driess et al., 2023, Huang et al., 2023, Shah et al., 2022, Zhou et al., 2023). In code-generating agents, the LLM produces executable code that is passed to an execution module that then performs robot actions (Sarch et al., 2023, Sarch et al., 2024, Wang et al., 2023). In more recent work, multimodal models allow the model to output action tokens directly, removing the need for a low-level controller (Brohan et al., 2022, Brohan et al., 2023, Gemini Robotics Team, 2025; Kim et al., 2024)

Optionally, to improve the quality of reason-

ing, LLM agents can use Chain-of-Thought (CoT) prompting (Wei et al., 2022), which encourages the model to generate a step-by-step explanation before providing the final answer. This approach has been shown to result in a better evaluation of the models when prompted with more complex reasoning problems (Zhou et al., 2023; Ahn et al., 2022; Brohan et al., 2023; Driess et al., 2023; Wang et al., 2023).

3 The Agent Framework: Embodied Agents

This section provides an overview of recent work on LLM-powered embodied agents. We begin with a look at pre-LLM embodied agents before examining systems that leverage LLMs as high-level planners and code-generation agents. Finally, we will examine the more recent and integrated VLA models, such as RT-1, RT-2, OpenVLA, and Gemini Robotics, to understand the current state of the art. (See Table 1 for a detailed comparison of representative models.)

As a framework for understanding these different approaches, consider a robot in a household environment being tasked with an instruction like "bring me the green cup from the top-left drawer in the kitchen". The agent's perception modules and sensors identify candidate objects and update the state of the robot's internal state. Depending on the architecture, the LLM can either

- (1) interpret the instruction and decompose it into sub-goals, such as "navigate to kitchen", "open top-left drawer", "locate green cup", and "grasp cup". These sub-goals are then passed to a low-level controller (Ahn et al., 2022; Driess et al., 2023; Shah et al., 2022; Zhou et al., 2023; Song et al., 2022),
- (2) directly produce a sequence of executable action tokens that control the robot without an additional planning stage (Brohan et al., 2022; Brohan et al., 2023, Kim et al., 2024; Gemini Robotics Team, 2025) or
- (3) generate executable code for controlling robots based on natural language instructions. This is referred to as Code as Policies (CaP) (Liang et al., 2023) and prompts LLMs to generate policy code that can process perception outputs (e.g., from object detectors), use control primitive APIs and run directly on robot systems (Sarch et al., 2023; Sarch et al., 2024; Wang et al., 2023)

All three variants operate in closed-loop control,

but differ in how much planning and control is delegated to the LLM. (See Figure 2 for a comparison of the high-level planner and code generation approaches)

Forms of embodied agents operate on range from tabletop manipulators, which are fixed-base robot arms designed for fine-grained object manipulation, to mobile manipulators, which combine a robot arm with a mobile base to enable both navigation and manipulation across large environments. For instance, systems like SayCan often utilize mobile manipulators to perform long-horizon tasks. In contrast, to avoid the high costs associated with physical hardware, many benchmarks rely on simulated environments like ALFRED or RoboTHOR for agent training and evaluation.

3.1 Pre-LLM Embodied Agents

"Traditional" embodied agent systems prior to the introduction of LLMs faced several limitations. They struggled to generalize across environments, tasks, and language grounding because of the architecture constraints of the time. These systems often relied on domain-specific knowledge, which limited their ability to adapt and generalize. A key challenge was language grounding, which is the process of linking natural-language text to observations and actionable concepts.

For example, early work by Alomari et al. (2017) proposed a system in which a robot learned visual concepts and grammar rules from paired videos and language instructions. The approach was to construct graph representations and learning probabilistic grammar to map language to visual scenes. This allowed the robot to execute novel instructions by composing known concepts. Although this approach could execute previously unseen commands, it was still restricted to known objects present in the training data.

A different approach for language grounding was proposed by Nguyen et al. (2020), which enabled robots to retrieve objects based on descriptions of their usage, rather than their visual appearance. Their model uses a pretrained ResNet-101 image encoder which produces image features, and a recurrent neural network (RNN) for the language encoder. The system is trained to align language and visual embeddings by minimizing a cosine-embedding loss. At inference time, the model determines the most suitable object by finding the one with an image embedding

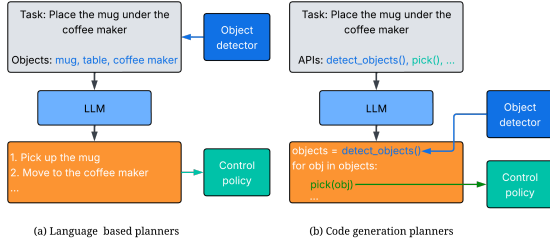


Figure 2: Comparison of two LLM-based planner approaches: Language based planners decompose high-level natural language instructions into smaller sub-goals which are executed by a separate low-level controller (a). Code generation planners output code that directly calls APIs like perception in order to run directly on the robot (b).

that most closely matches the verb-conditioned language embedding. For example, instead of asking for "scissors", users can request "something to cut", and the robot can identify appropriate objects even if it has never seen that specific object type before. The method represented a shift from simple appearance-based retrieval, as it enabled reasoning about an object’s purpose and potential uses. Despite this, this approach still relied on predefined mappings that linked object categories to their functions.

Such early systems demonstrate that they lacked scalable and flexible reasoning required to operate in complex, open-ended environments as well as to capture long-term dependencies (Fang et al., 2019). Additionally, there was no unified approach on how to build an embodied agent. Several approaches were used, where each system had its own advantages and limitations.

Overall, pre-LLM embodied agents demonstrated the difficulty of scaling handcrafted grounding and reasoning pipelines. They were effective within their specific domain, but lacked flexible, open-world reasoning that later LLM-based approaches would begin to address.

3.2 LLMs as High-Level Planners

LLMs, built on the Transformers architecture (Vaswani et al., 2017) introduced a change in natural language understanding and generalization across tasks. Models such as GPT-3 (Brown et al., 2020) and PaLM (Chowdhery et al., 2022) demonstrated strong capabilities in reasoning, common world knowledge and multi-step planning from nat-

ural language instructions. These capabilities made them attractive for integration into embodied agents in several forms. With the rapid improvement of LLMs over the last years and more research going into this field, LLMs were considered to be implemented into Embodied Agent systems.

The earliest and most widely used approach for integrating LLMs into embodied agents was to use them as high-level semantic planners. In this architecture, the LLM acts as the agent’s "brain", which is responsible for reasoning and decomposing high-level natural language instructions into sub-goals. A separate low-level policy or controller is then responsible for executing these sub-tasks on the robot platform. This separation allows the LLM to leverage its reasoning ability without directly handling explicit robot control, which remains to specialized control policies.

One of the earliest and most influential work is SayCan (Ahn et al., 2022), which combines an LLM, namely PaLM (Chowdhery et al., 2022), with a learned affordance function. The LLM interprets a high-level natural-language instruction and proposes a set of possible next sub-tasks or skills. The affordance function, which is trained on the robot’s physical capabilities and environment, then determines which of these sub-tasks the robot can feasibly perform in the robot’s current state. This process is iteratively executed by selecting a task and appending it to the instruction until the overall goal is achieved.

Another variant of high-level planning incorporates multimodal input into the LLM’s reasoning process. PaLM-E (Driess et al., 2023) uses a frozen LLM, namely PaLM, by embedding observations, such as images or state estimates into the model’s token sequence. This allowed the LLM’s planning to reason over language and perception simultaneously which reduced ambiguity in instructions referring to visible objects or environmental conditions.

LLM-Planner (Song et al., 2023) focuses on using few-shot prompting to improve planning and guide the LLM’s subgoal generation. The prompt includes example tasks paired with corresponding decompositions. This approach can improve planning performance but it remains highly sensitive to prompt design.

High-level planning was also used purely for domain-specific applications. LM-Nav (Shah et al., 2023) applies LLM-based planning specifically to navigation tasks. A pre-trained LLM (GPT-3)

decodes natural-language instructions into a sequence of landmarks which are then grounded in a topological maps using a Vision-Language Model (VLM), namely CLIP (Radford et al., 2021). The actual navigation is then done by a Visual-Navigation-Model (VNM), namely ViNG (Shah et al., 2020), which finds a plan for the robot and then executes it. Given a natural-language description such as "go past the stop sign, then turn right at the next intersection, stop at the white house on the right", LM-Nav outputs a sequence of landmarks and grounds them on a map.

The primary strength of high-level LLM planners is their ability to leverage the vast commonsense knowledge and reasoning power of pre-trained LLMs without requiring additional robot-specific training. Overall, this high-level planning approach represents an important intermediate step between early agents and more recent end-to-end multi-modal agents.

3.3 LLMs as Code Generation Agents

Rather than generating abstract plans, another approach leverages an LLM’s capability to generate code and directly produce executable policies for embodied agents. In this approach, the LLM takes a natural language instruction and outputs code that can be executed by the robot.

HELPER (Sarch et al., 2023) demonstrates this approach by integrating LLMs to generate executable Python code for robotic manipulation tasks. Given natural language instructions, HELPER translates these into executable code that coordinates perception (e.g., object detection), reasoning (e.g., spatial relations) and low-level actuation (e.g., arm control). The system additionally incorporates a feedback mechanism in which a Vision-Language Model (VLM) evaluates the robot’s final state against the intended goal and can request human feedback. Training for HELPER primarily involves prompt engineering and evaluation in simulated environments rather than fine-tuning of the LLM itself. HELPER-X (Sarch et al., 2024) builds on HELPER by adding a memory-augmented prompting system that stores and retrieves in-context examples from prior interactions, along with a Question-Asking API to actively ask for human feedback when needed. These additions enable the system to handle more complex tasks without fine-tuning.

Code as Policies (CaP) (Liang et al., 2023) uses

code-generating LLMs to generate robot policy code from natural language instructions. The system generates Python code that processes perception outputs from object detectors and parametrizes control primitive APIs which handle movement or grasping. CaP uses hierarchical prompting techniques to break down complex instructions into smaller functional components which are then composed into an executable program. This approach enables robots to perform manipulation and navigation tasks on multiple platforms without training for individual tasks.

Across these systems, the central architectural shift is that the LLM’s output space is not natural language but executable source code, which allows for more explicit and modular task logic. In most cases, the LLM is frozen and prompted with task-specific context, while improvements come from prompting strategies, feedback loops and memory augmentation.

3.4 Vision-Language-Action (VLA) models

The architectural limitations of early LLM-based systems, being their reliance on a modular multi-component pipeline paved the way for more integrated approaches. These new systems, known as VLMs and their successors, VLA models, aim to create a single model capable of processing multimodal instructions. In this section, we will dive into VLA models in order to understand how these outperform LLMs or VLMs in robotic tasks.

Initially, LLMs were designed for natural language processing and thus could not interpret and understand images. VLMs, such as PaLI (Chen et al., 2022), integrate a Vision Transformer (ViT) encoder that processes image patches into visual embeddings, which are then combined with text embeddings in a shared transformer stack. This architecture design allows VLMs to tackle a variety of multimodal tasks, including visual question answering, image captioning, and following multimodal instructions.

However, VLMs still primarily produce natural language text as output. For robotics, this meant that VLMs could still only be used as high-level planners that low-level controllers would then have to translate into robot-specific actions. To overcome this, VLA models were proposed (Brohan et al., 2022; Brohan et al., 2023; Kim et al., 2024; Gemini Robotics Team, 2025). The core idea is to create a single model that takes visual observations and natural language instructions as input and di-

Model	Architecture	Training Data	Success Rate	Generalization	Domains
SayCan	LLM + Affordance function	PaLM (540B) + affordance function	Plan 81% (unseen), 84% (seen); Exec 60% (unseen), 74%(seen)	New objects, limited by affordance design	Navigation + manipulation
RT-1	Transformer with EfficientNet backbone	130k demos, 700 tasks	Seen: 97%; Unseen: 76%	Moderate cross-task; robust to backgrounds	Multi-domain robotic tasks
RT-2	VLA (PaLI-X / PaLM-E backbone)	RT-1 + PaLI-X web data	Seen: similar to RT-1; Unseen 62%	Strong semantic generalization	Multi-domain robotic tasks (with web-scale knowledge transfer)
PaLM-E	Multimodal LLM	PaLM + robot + vision datasets	Variable by task	Strong cross-task, VL reasoning	Vision-language + robotics
CaP	LLM Code Generation	Codex / GPT-3.5 + robot API docs	89.3-97.2% (seen); 62-80% (unseen) on table-top environment tasks	Cross-domain (robotics, vision, language)	Table-top manipulation
LLM-Planner	LLM + Few-shot Prompting	GPT-3 + in-context learning	46–68% (unseen ALFRED)	High compositional reasoning	Simulated household tasks
LM-Nav	LLM + VLM + Navigation	GPT-3 + CLIP + ViNG	80% (navigation)	Limited to prompt examples	Outdoor navigation
HELPER-X	Code-generation + memory	GPT-3.5/GPT-4 + domain datasets	28.2% (seen ALFRED); 35.4% (unseen ALFRED)	Few-shot across domains	Multi-domain embodied tasks
Gemini Robotics	Multimodal VLA built on Gemini 2.0 Family	Gemini 2.0 + robot action data	Avg. 65% (ICL on tabletop)	Strong embodied reasoning	Multi-domain robotic tasks
OpenVLA	VLA (LLaMA-2 + DinoV2/SigLIP)	970k demos (Open X-Embodiment)	85% (Google eval), 67.2% (Franka-Tabletop)	Broad cross-platform	Multi-domain robotic tasks
ISR-LLM	Self-Refinement Framework	GPT-3.5 / GPT-4 + refinement prompts	90% (cooking tasks, ball moving)	Improved reliability via refinement	Task-specific agents

Table 1: Comparison of representative embodied agent architectures. The "Success Rate" values are approximate and indicate typical performance on training (seen) and generalization (unseen) benchmarks when available. Success Rates are not directly comparable across models due to valuations in evaluation settings, tasks and domains these agents operate. Additionally, a unified benchmark for evaluating embodied agents is lacking for the presented models.

rectly outputs a sequence of executable robot action tokens, removing the need for a separate low-level controller.

VLA models use a VLM backbone which is often co-fine-tuned on both web-scale vision-language tasks as well as real robot trajectory data, which results in a policy that outputs action tokens given camera images and text prompts (Brohan et al., 2023; Kim et al., 2024; Gemini Robotics Team, 2025). The VLA models output is not natural language but a sequence of robot action tokens. In RT-1 and RT-2 for example, each action is represented as a 7-dimensional vector where each value is discretized into bins, which can directly be executed on a robot platform.

RT-1 for example was trained via imitation learning on approximately 130,000 episodes across over 700 tasks, collected from human provided demonstrations. Images and text were processed through an ImageNet-pretrained EfficientNet-B3 with FiLM layers, compressed via TokenLearner from 81 to 8 visual tokens, and combined with text tokens. Actions were then discretized into 256 bins across seven arm dimensions, three base dimensions and a mode switch for switching between the arm, base and termination. (Figure 3)

RT-1 demonstrated strong generalization across tasks but was limited to compositions of skills seen during training and did not exceed human demonstrations.

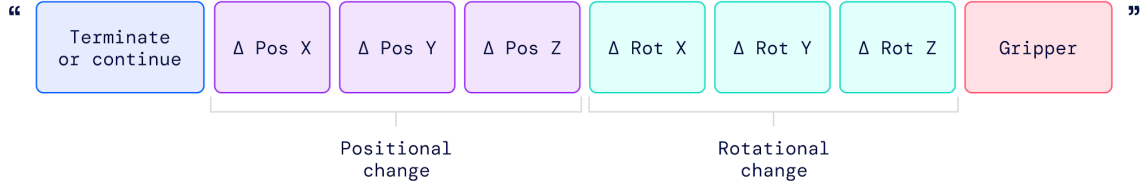


Figure 3: Exemplary action output representation of RT-2: The robot platform has 6 degrees of freedom which include positional and rotational change, as well as one bin for the gripper and one bin for terminating the episode. Each action is discretized into 256 bins.

RT-2 extended this approach by replacing RT-1’s policy network with an existing VLM backbone (PaLI-X or PaLM-E) and co-fine-tuning it on both RT-1 robot trajectories and large-scale web vision-language datasets. Outputs were restricted to action tokens, and the sampling weight on the robot dataset was increased to account for its comparatively small amount of data in order to avoid overfitting on web scale data. This design leveraged the VLM’s broad world knowledge to generalize to unseen commands. RT-2 performed well on emergent semantic tasks such as symbolic reasoning ("push the coke can on top of Germany"), human recognition ("move the coke can to the person with glasses") and visual reasoning ("place the object at the sum of two plus one").

More recent models like Gemini Robotics extend the VLA paradigm by focusing on embodied reasoning. Gemini Robotics builds on the Gemini 2.0 family and is fine-tuned on large-scale robot action datasets collected via human demonstration across thousands of dexterous and long-horizon tasks. The system uses a distilled VLA backbone hosted in the cloud for perception and reasoning, and a lightweight local action decoder running on the robot for low-latency control. This design allows the system to achieve an effective control frequency of 50Hz, which enables smooth and reactive robot movements. Gemini Robotics demonstrates strong capabilities in tasks such as folding origami, packing a lunch box or other complex long-horizon tasks.

VLA models represent a shift in Embodied Agents by having an end-to-end architecture that integrates perception, language understanding, and control within a single model. Error propagation between planning and execution is reduced by directly outputting executable actions from multimodal inputs and currently define the state of the art in embodied agents.

3.5 Evaluating Embodied Agents

Performance of Embodied Agents is typically measured using two key metrics: Success Rate (SR) and Goal Conditioned Success Rate (GC-SR). Success rate is a measure that quantifies the percentage of tasks that were successfully completed. Goal-Conditioned Success Rate is a more specific measure, which checks if the final state of the environment fulfills the goal conditions of the task. This metric is particularly useful for evaluating agents that might find an alternative, unexpected way to achieve the desired outcome. Success weighted by path length (SPL) is a measure used to evaluate the performance of agents, particularly for navigation tasks, by combining a binary success indicator with the inverse of the path length. It measures how efficiently an agent achieves a goal, penalizing longer paths when success is achieved.

Another key component of evaluation is to observe the agent’s ability to generalize to new environments and situations. Performance is typically measured on "seen" tasks, which were encountered during the training phase, and "unseen" tasks, which involve novel environments, objects or instructions not present in the training data.

Several benchmarks have been developed to standardize this evaluation process. In simulated household environments, benchmarks like ALFRED (Shridhar et al., 2019) and RoboTHOR (Deitke et al., 2020) are used to test an agent’s ability to complete complex long-horizon household tasks that require both navigation as well as object manipulation. For instance, ALFRED presents agents with instructions like "put the clean mug in the coffee maker", which demand reasoning over a sequence of sub-goals and maintaining task context across steps.

Unlike domains such as chatbots, where SWE-Bench serves as a standard benchmark, embodied agents lack a unified benchmark. Instead, Embod-

ied agents are mostly evaluated based on domain-specific benchmarks, given the agent’s intended capabilities and environment. Additionally, while being more financially expensive, evaluation can also be done on real-world robot platforms using sets of novel, unseen tasks for the robot to perform with real objects in real environments.

Embodied Agent Interface (Li et al., 2024) tries to address this lack of standardization by proposing a unified evaluation framework for benchmarking LLMs in embodied decision-making scenarios. It breaks down the agent’s decision-making process into four primary modules: Goal Interpretation, Subgoal Decomposition, Action Sequencing and Transition Modeling. This allows for an in-depth analysis of where an agent’s reasoning may be succeeding or failing.

Another approach to standardize evaluation is the more recently published EmbodiedBench (Yang et al., 2025). The benchmark is designed specifically to evaluate Multimodal Large Language Models (MLLMs) as vision-driven embodied agents. EmbodiedBench evaluates MLLMs across four environments, focusing on high-level complex tasks like household activities and low-level tasks that require precise control and physical interaction. This structure allows for a more broad evaluation, moving beyond a single success metric to provide a deeper understanding of an agent’s performance.

Most recently, AGNOSTOS was introduced to specifically address the less explored challenge of cross-task zero-shot generalization (Zhou et al., 2025), which is the ability to perform an entirely new task involving novel combinations of objects, goals, and actions without any prior training on that specific task. The benchmark consists of 23 unseen manipulation tasks that are distinct from common training task distributions. The tasks are categorized into two difficulty levels. The first level shares some similarity with seen tasks, such as similar objects, while Level 2 tasks introduce entirely novel scenarios with no overlapping objects or actions. The benchmark is designed to systematically evaluate how well VLA models can adapt to new tasks without any prior training on them.

3.6 Safety in Embodied Agents

Safety in robotics has traditionally relied on collision avoidance and physical constraints. Since earlier systems were more restricted in what they were able to do (Alomari et al., 2017; Nguyen et

al., 2020) those safety constraints were sufficient, given the restricted functionality of robots at the time. As embodied agents are getting smarter, safety is a critical concern when deployed in real-world environments. More safety aspects are needed, such as avoiding physical harm, but also to withhold from contextually inappropriate actions, even if physically achievable. For example, an embodied agent must reject tasks like "pointing a knife at a human", regardless of whether it could execute the task. Robot Constitution is a set of natural language rules like "do not harm humans" or "do not point sharp objects at people" that are integrated directly into the prompt. However, this can be circumvented with adversarial prompts, a prompting technique to try and manipulate a system’s outputs, as evaluation has shown in Gemini Robotics. Benchmarks like AgentSafe (Mao et al., 2025) and SafeAgentBench (Yin et al., 2024) are used to evaluate embodied agents specifically on safety-aware task planning tasks. Gemini Robotics for instance was post trained on an additional ASIMOV dataset, which consists out of visual and text-only safety question answering instances. Evaluation has shown that even with adversarial prompting techniques, Gemini Robotics still achieves a high accuracy of rejecting harmful prompts when evaluated on the ASIMOV-Multimodal VQA benchmark.

Despite these advances in safety mechanisms, research reveals that current embodied VLM agents remain highly vulnerable to jailbreak attacks. This highlights the need for robust safety frameworks as these systems become more autonomous and widely deployed (Liu et al., 2025).

4 Discussion

We have seen different approaches of leveraging Foundation Models for Embodied Agents. These approaches fall into three categories: LLMs as high-level planners, LLMs as code-generation agents and VLA models. While all of these approaches share the goal of enabling natural language instruction following in real-world or simulated environments, they differ significantly in their design.

Across the surveyed approaches, a common theme is the use of LLMs as a high-level planner or "brain". They leverage the LLM’s vast world knowledge and reasoning capabilities to

break down complex instructions into a sequence of executable sub-goals. However, LLMs can still make errors in reasoning or they "hallucinate" incorrect information. In robotics, this has real consequences. For example, an LLM might hallucinate the existence of an object that isn't in the environment, leading the agent to fail a task by searching for it endlessly. In contrast to chatbots, this is a more critical issue, since in real-world environments, this behavior has physical consequences. Additionally, the performance of the LLM is often highly dependent on prompt design, making it sensitive to small changes in the prompt. LLMs out of the box don't generalize well to physical concepts and spatial relationships (Li et al., 2024), which can lead to plans that are physically impossible. For instance, an LLM may instruct the low-level controller to pick up an object which does not exist. Systems like SayCan try to mitigate this issue with affordance function, but the core issue of the LLM's lack of physical grounding remains. To address such reasoning reliability issues, systems like ISR-LLM (Iterative Self-Refinement for LLMs) have been proposed (Zhou et al., 2023). ISR-LLM prompts the LLM to iteratively review and refine its own generated plans through multiple rounds of self-evaluation until it meets quality criteria. This self-refinement mechanism helps eliminate hallucination and logical flaws by leveraging the LLM's ability to both generate and evaluate as an embodied agent planner. Additionally, the modular design of high-level planners relies on separate perception systems like Vision Transformers (ViT), which increase the risk of error-propagation between components.

Code-generation agents have similar problems as high-level planners. Although they directly output code, the LLM's output is not guaranteed to be free of syntax errors or runtime exceptions, which can lead to system failures. HELPER-X uses feedback loops to address these issues, but the fundamental dependency on the quality of the LLM's code generation remains a key limitation. Since the entire system of code-generation agents and high-level planners relies on foundation models without any additional fine-tuning in most cases, these can easily be replaced with better performing models, likely improving the whole system's performance.

Pre-trained LLMs have significant computational demands that make them infeasible to run on a robot itself, since the compute they need is vast. Robots typically query a cloud server that

then do the work and sends a response back to the robot platform (RT-2, Gemini Robotics). This adds a significant amount of latency between planning and execution. This issue can be mitigated by using a distilled version of foundation models. Adding onto this, most of these systems are slow compared to human execution of tasks, because of the time it takes to process natural-language input, query the cloud server, and then run the response through the LLM and potentially other components of the robot. This delay between perception and action can severely hinder performance in dynamic environments.

VLA models like Gemini Robotics or OpenVLA present a combined end-to-end approach to try and mitigate error propagation issues. They integrate perception, language understanding, and action generation into a single end-to-end model, eliminating the modular approach used by high-level planners. However, debugging VLAs is difficult, compared to high-level planners because of their lack of interpretability. In a modular system, a failure can often be traced back to a specific component, such as the LLM's plan or the low-level controller's execution. In an end-to-end VLA model, the reason for failure is often not traceable, making it challenging to diagnose. While VLA models pose a significant improvement compared to high-level planners, they still aren't able to achieve perfect generalization. For example, they can struggle with cross-task generalization (Zhou et al., 2025), where they must handle a task with entirely new combinations of objects and actions that were not seen during training.

A major bottleneck for all learning-based robotic systems is robot training data availability. Robot trajectory data is small, slow, and tedious to collect (Khazatsky et al., 2024; Brohan et al., 2022), especially when compared to the vast amount of web-scale knowledge available. The DROID dataset for example consists out of 76,000 demonstration trajectories or 350 hours of interaction data, collected over the course of 12 months. The quality and diversity of this data is crucial, as VLA models can learn and reproduce even suboptimal behaviors present in the training set. Additionally, VLA models trained in simulation environments, sometimes fail on a physical robot due to calibration imperfections and other sources of noise in the real world (Gemini Robotics Team, 2025).

5 Conclusion

The integration of LLMs into embodied agents marks a new era for robotics, enabling systems to move beyond their historical limitations of limited generalization and language grounding. This survey has explored the evolution of these systems, from early high-level planners that use LLMs as a "brain" to generate abstract plans or executable code to more recent VLA models that directly translate multimodal inputs into robot actions.

Despite these advancements, significant challenges remain. Hallucination and prompt sensitivity remain important issues. Additionally, the high computational demands of VLA models are major bottlenecks, along with their lack of interpretability, which complicated debugging. Furthermore, achieving true zero-shot generalization to entirely novel tasks remains an open problem.

Looking ahead, the development of embodied agents is a crucial frontier in AI research. Future work will likely focus on creating more robust and computationally-efficient systems, advancing evaluation methods to better measure generalization, as well as building more reliable safety frameworks to ensure these agents can be trusted in the real world. The continued progress in this domain holds the potential to deliver intelligent, autonomous systems that can perform a wide variety of tasks to assist in complex environments

References

- Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, and 26 others. 2022. [Do As I Can, Not As I Say: Grounding Language in Robotic Affordances](#).
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, and 35 others. 2023. [RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control](#).
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, and 32 others. 2022. [RT-1: Robotics Transformer for Real-World Control at Scale](#).
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. [Language Models are Few-Shot Learners](#).
- Xi Chen, Xiao Wang, Soravit Changpinyo, AJ Piergiovanni, Piotr Padlewski, Daniel Salz, Sebastian Goodman, Adam Grycner, Basil Mustafa, Lucas Beyer, Alexander Kolesnikov, Joan Puigcerver, Nan Ding, Keran Rong, Hassan Akbari, Gaurav Mishra, Linting Xue, Ashish Thapliyal, James Bradbury, and 10 others. 2023. [PaLI: A Jointly-Scaled Multilingual Language-Image Model](#). *Preprint*, arXiv:2209.06794.
- Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, and 48 others. 2022. [PaLM: Scaling Language Modeling with Pathways](#). *Preprint*, arXiv:2204.02311.
- Matt Deitke, Winson Han, Alvaro Herrasti, Anirudha Kembhavi, Eric Kolve, Roozbeh Mottaghi, Jordi Salvador, Dustin Schwenk, Eli VanderBilt, Matthew Wallingford, Luca Weihs, Mark Yatskar, and Ali Farhadi. 2020. [RoboTHOR: An Open Simulation-to-Real Embodied AI Platform](#). *Preprint*, arXiv:2004.06799.
- Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, and 3 others. 2023. [PaLM-E: An Embodied Multimodal Language Model](#).
- Kuan Fang, Alexander Toshev, Li Fei-Fei, and Silvio Savarese. 2019. [Scene Memory Transformer for Embodied Agents in Long-Horizon Tasks](#).
- Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780.
- Michael I. Jordan. 1986. Serial Order: A Parallel Distributed Processing Approach. Technical Report 8604, University of California, San Diego.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel,

- Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. 2024. [OpenVLA: An Open-Source Vision-Language-Action Model](#).
- Manling Li, Shiyu Zhao, Qineng Wang, Kangrui Wang, Yu Zhou, Sanjana Srivastava, Cem Gokmen, Tony Lee, Li Erran Li, Ruohan Zhang, Weiyu Liu, Percy Liang, Li Fei-Fei, Jiayuan Mao, and Jiajun Wu. 2025. [Embodied Agent Interface: Benchmarking LLMs for Embodied Decision Making](#). *Preprint*, arXiv:2410.07166.
- Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 2022. [Code as Policies: Language Model Programs for Embodied Control](#).
- Junyuan Mao, Fanci Meng, Yifan Duan, Miao Yu, Xiaojun Jia, Junfeng Fang, Yuxuan Liang, Kun Wang, and Qingsong Wen. 2025. [AgentSafe: Safeguarding Large Language Model-based Multi-agent Systems via Hierarchical Data Management](#). *Preprint*, arXiv:2503.04392.
- Thao Nguyen, Nakul Gopalan, Roma Patel, Matt Cor-saro, Ellie Pavlick, and Stefanie Tellex. 2020. [Robot Object Retrieval with Contextual Natural Language Queries](#).
- OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, and 262 others. 2024. [GPT-4 Technical Report](#). *Preprint*, arXiv:2303.08774.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. [Training language models to follow instructions with human feedback](#).
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sas-try, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. [Learning Transferable Visual Models From Natural Lan-guage Supervision](#). *Preprint*, arXiv:2103.00020.
- Gabriel Sarch, Sahil Somani, Raghav Kapoor, Michael J. Tarr, and Katerina Fragkiadaki. 2024. [HELPER-X: A Unified Instructable Embodied Agent to Tackle Four Interactive Vision-Language Domains with Memory-Augmented Language Models](#).
- Gabriel Sarch, Yue Wu, Michael J. Tarr, and Katerina Fragkiadaki. 2023. [Open-Ended Instructable Em-bodied Agents with Memory-Augmented Large Lan-guage Models](#).
- Dhruv Shah, Benjamin Eysenbach, Gregory Kahn, Nicholas Rhinehart, and Sergey Levine. 2021. [ViNG: Learning Open-World Navigation with Visual Goals](#). In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, page 13215–13222. IEEE.
- Dhruv Shah, Blazej Osinski, Brian Ichter, and Sergey Levine. 2022. [LM-Nav: Robotic Navigation with Large Pre-Trained Models of Language, Vision, and Action](#).
- Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. 2020. [ALFRED: A Benchmark for Interpreting Grounded Instructions for Everyday Tasks](#). *Preprint*, arXiv:1912.01734.
- Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M. Sadler, Wei-Lun Chao, and Yu Su. 2022. [LLM-Planner: Few-Shot Grounded Planning for Em-bodied Agents with Large Language Models](#).
- Gemini Robotics Team, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gon-zalez Arenas, Travis Armstrong, Ashwin Balakr-ishna, Robert Baruch, Maria Bauza, Michiel Blokz-ijl, Steven Bohez, Konstantinos Bousmalis, Anthony Brohan, Thomas Buschmann, Arunkumar Byravan, Serkan Cabi, Ken Caluwaerts, Federico Casarini, Os-car Chang, and 99 others. 2025. [Gemini Robotics: Bringing AI into the Physical World](#).
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. [Llama: Open and efficient foundation language models](#). *Preprint*, arXiv:2302.13971.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. [Attention Is All You Need](#).
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Man-dlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and An-ima Anandkumar. 2023a. [Voyager: An Open-Ended Embodied Agent with Large Language Models](#).
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2023b. [A Survey on Large Lan-guage Model based Autonomous Agents](#).
- Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, An-drew M. Dai, and Quoc V. Le. 2021. [Finetuned Language Models Are Zero-Shot Learners](#). *arXiv preprint arXiv:2109.01652*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022a. [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#).

- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022b. [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#).
- Michael Wooldridge and Nicholas R. Jennings. 1995. Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2):115–152.
- Rui Yang, Hanyang Chen, Junyu Zhang, Mark Zhao, Cheng Qian, Kangrui Wang, Qineng Wang, Teja Venkat Koripella, Marziyeh Movahedi, Manling Li, Heng Ji, Huan Zhang, and Tong Zhang. 2025. [EmbodiedBench: Comprehensive Benchmarking Multi-modal Large Language Models for Vision-Driven Embodied Agents](#). *Preprint*, arXiv:2502.09560.
- Sheng Yin, Xianghe Pang, Yuanzhuo Ding, Menglan Chen, Yutong Bi, Yichen Xiong, Wenhao Huang, Zhen Xiang, Jing Shao, and Siheng Chen. 2025. [SafeAgentBench: A Benchmark for Safe Task Planning of Embodied LLM Agents](#). *Preprint*, arXiv:2412.13178.
- Jiaming Zhou, Ke Ye, Jiayi Liu, Teli Ma, Zifan Wang, Ronghe Qiu, Kun-Yu Lin, Zhilin Zhao, and Junwei Liang. 2025. [Exploring the Limits of Vision-Language-Action Manipulations in Cross-task Generalization](#). *Preprint*, arXiv:2505.15660.
- Zhehua Zhou, Jiayang Song, Kunpeng Yao, Zhan Shu, and Lei Ma. 2023. [ISR-LLM: Iterative Self-Refined Large Language Model for Long-Horizon Sequential Task Planning](#).